

PYTHON FOR QUANT TRADERS · PART VII (ตัวอย่าง)

Stat Arb — จับมือทำตั้งแต่ต้นจนจบ

บทที่ 37: สแกนหาหุ้น — แล้วเจอผี 12 ตัว

ตัวอย่างรูปแบบของ Part VII · ยังไม่ใช่ฉบับสมบูรณ์

นี่คือตัวอย่างเพื่อดูรูปแบบ

Part 0–VI เป็นบทเรียน — สอนทีละเครื่องมือ · Part VII เป็น **สมุดบันทึกการทำงานจริง** — ทำตามได้ที่ละขั้นจนได้ระบบที่รันจริง

ทุกบทใน Part VII มีโครงเดียวกัน 5 ส่วน: 🎯 เป้าของเซสชัน → 🖐️ ลงมือ → 🛑 จุดตัดสินใจ → ✅ เช็คความถูกต้อง → 📁 สถานะโปรเจกต์ · บทที่ยกมาให้ดูนี้คือแกนของทั้ง Part

กฎเหล็กของ Part นี้: ไม่สอนทฤษฎีซ้ำสักระลอก — cointegration อยู่บทที่ 14 · multiple testing อยู่ 28.5 · ต้นทุนอยู่บทที่ 25 · ตรงไหนต้องใช้ก็ลิงก์กลับ ไม่เขียนใหม่

บทที่ 37: สแกนหาคู่ — แล้วเจอผี 12 ตัว

🎯 เป้าของเซสชันนี้

จบบทนี้คุณจะมี **shortlist** ของคู่ที่ผ่านเกณฑ์จริง อยู่ในไฟล์ พร้อมเหตุผลกำกับทุกคู่ที่ถูกตัดออก

และจะเข้าใจสิ่งที่สำคัญกว่านั้น: **ทำไม shortlist ที่ดีถึงต้องสั้น** — ถ้าสแกนแล้วเจอคู่ที่น่าสนใจเป็นสิบ นั่นไม่ใช่โชคดี นั่นคือเกณฑ์ยังหลวมอยู่

📖 **อ่านว่า:** บทที่ 14 สอนไปแล้วว่าทดสอบ cointegration ของคู่หนึ่ง ยังไง · บทนี้ต่างกันตรงที่เราจะทดสอบ **ทุกคู่ที่เป็นไปได้พร้อมกัน** — และการทำแบบนี้เปลี่ยนความหมายของ p-value ไปทั้งหมด · นี่คือช่องว่างระหว่าง "ทำ cointegration test เป็น" กับ "หาคู่เองเป็น"

👉 ขั้นที่ 1 — สร้างคู่ทั้งหมดที่เป็นไปได้

จบบทที่ 36 คุณจะมี `prices` เป็น DataFrame เดียว index เป็นเวลา คอลัมน์เป็นเหรียญ · บทนี้เริ่มจากตรงนั้น

```
import itertools
import numpy as np
import pandas as pd
from statsmodels.tsa.stattools import coint

# prices มาจากบทที่ 36 — ที่นี่ใช้ข้อมูลจำลองที่ "ปลูก" คู่จริงไว้ 3 คู่
# เพื่อให้เราวัดได้ว่าเครื่องกรองจับได้กี่คู่ และปล่อยผีผ่านมากี่ตัว
prices = make_universe(seed=7)          # 30 เหรียญ · 730 วัน

pairs = list(itertools.combinations(prices.columns, 2))
print(f"เหรียญ {prices.shape[1]} ตัว → {len(pairs)} คู่")
print(f"ถ้าเพิ่มเป็น 100 เหรียญ → {100*99//2:,} คู่")
```

► OUTPUT

```
เหรียญ 30 ตัว → 435 คู่
ถ้าเพิ่มเป็น 100 เหรียญ → 4,950 คู่
```

🤖 `itertools.combinations` — และตัวเลขที่โตเร็วกว่าที่คิด

```
list(itertools.combinations(["A","B","C"], 2))
# → [("A","B"), ("A","C"), ("B","C")]   ไม่มี ("B","A") ซ้ำ
```

```
# จำนวนคู่ =  $n(n-1)/2$  → โทแบบกำลังสอง
# 30 เหรียญ → 435 คู่
# 100 เหรียญ → 4,950 คู่
# 500 เหรียญ → 124,750 คู่
```

1. combinations ให้คู่ที่ไม่สนลำดับและไม่ซ้ำ — ต่างจาก permutations ที่จะได้ทั้ง (A,B) และ (B,A) . สำหรับ pair trading คู่ A-B กับ B-A คือคู่เดียวกัน (ต่างแค่ใครเป็นขา long)
2. ตัวเลขโทแบบกำลังสอง — เพิ่มเหรียญ 3 เท่า คู่เพิ่ม ~10 เท่า . เรื่องนี้ไม่ใช่แค่ปัญหาความเร็ว มันคือปัญหาทางสถิติ ที่จะเห็นในขั้นถัดไป
3. ที่ต้องครอบด้วย list() เพราะ combinations คือน generator (หัวข้อ 7.7) — เดินได้รอบเดียว . เราต้องวนหลายรอบจึงแปลงเป็น list ก่อน

👉 ขั้นที่ 2 — ทดสอบทุกคู่

```
# ใช้ coint() ตัวเดียวกับบทที่ 14 — แคว้นทุกคู่
rows = []
for a, b in pairs:
    _, pvalue, _ = coint(np.log(prices[a]), np.log(prices[b]))
    rows.append((a, b, pvalue))

res = (pd.DataFrame(rows, columns=["a", "b", "pvalue"])
      .sort_values("pvalue")
      .reset_index(drop=True))

print(f"ผ่านเกณฑ์ปกติ p < 0.05 : {(res.pvalue < 0.05).sum()} คู่")
print(res.head(6).to_string(index=False,
                             formatters={"pvalue": "{:.2e}".format}))
```

► OUTPUT

```
ผ่านเกณฑ์ปกติ p < 0.05 : 15 คู่
   a      b  pvalue
COIN05 COIN06 1.23e-08
COIN00 COIN01 1.24e-05
COIN10 COIN11 2.58e-03
COIN03 COIN06 8.85e-03
COIN03 COIN05 9.09e-03
COIN13 COIN29 1.71e-02
```

✅ เช็คว่ามาถูกทาง

สามอันดับแรกคือ COIN05/06 · COIN00/01 · COIN10/11 — ตรงกับคู่ที่เราปลูกไว้ทั้ง 3 คู่พอดี · เครื่องมือทำงานถูก

ถ้าของคุณไม่ตรงให้ย้อนไปเช็คค่าใช้ `np.log()` ครบทั้งสองขาไหม — cointegration ของราคาดิบกับของ log price ให้คำตอบคนละอย่าง

● หยุดตรงนี้ก่อน — "15 คู่" เป็นข่าวดีหรือข่าวร้าย

สัญญาณบอกว่าคุณดี: หาไป 435 คู่ เจอ 15 คู่ที่ผ่านเกณฑ์ทางสถิติที่ยอมรับกันทั้งวงการ

แต่เรารู้ความจริง — เราปลุกคู่จริงไว้แค่ 3 คู่ · อีก 12 คู่ที่เหลือคือผี · 80% ของ shortlist เป็นเสียงรบกวน

และในงานจริงคุณไม่รู้ว่าคุณอันไหนจริงอันไหนผี — คุณจะเห็นแค่ตาราง 15 แถวที่หน้าตาน่าเชื่อถือเท่ากันหมด

● จุดตัดสินใจที่ 1 — ทำไมถึงมีผี 12 ตัว

คำตอบอยู่ในนิยามของ $p < 0.05$ เอง: "โอกาสที่จะเห็นผลแบบนี้โดยบังเอิญ ทั้งที่ไม่มีความสัมพันธ์จริง = 5%"

ทดสอบ 1 คู่ → พลาด 5% · ทดสอบ 435 คู่ → คาดว่าจะพลาดราว $435 \times 5\% \approx 22$ คู่ · ไม่ใช่ความผิดพลาด แต่คือสิ่งที่เกณฑ์นี้สัญญาไว้ตั้งแต่แรก

พิสูจน์แบบไม่มีทางเถียง — รันบนข้อมูลที่ไม่มีคู่ไหนสัมพันธ์กันเลยแม้แต่คู่เดียว:

```
# random walk อีสรุล้วน ๆ 40 ตัว — ไม่มีความสัมพันธ์ใด ๆ ทั้งสิ้น
rng = np.random.default_rng(0)
noise = np.cumsum(rng.normal(0, 1, (40, 500)), axis=1) + 100

noise_pairs = list(itertools.combinations(range(40), 2))
hits = sum(1 for i, j in noise_pairs
           if coint(noise[i], noise[j])[1] < 0.05)

print(f"ลึนทรัยพ์ 40 ตัว = {len(noise_pairs)} คู่ · ไม่มีคู่ไหนสัมพันธ์กันจริงเลย")
print(f"แต่ผ่าน cointegration ที่ p<0.05: {hits} คู่ ({hits/len(noise_pairs):.1%})")
print(f"Bonferroni: ต้องใช้ p < {0.05/len(noise_pairs):.2e} แทน")
```

► OUTPUT

```
ลึนทรัยพ์ 40 ตัว = 780 คู่ · ไม่มีคู่ไหนสัมพันธ์กันจริงเลย
แต่ผ่าน cointegration ที่ p<0.05: 41 คู่ (5.3%)
Bonferroni: ต้องใช้ p < 6.41e-05 แทน
```

ตัวเลขที่ควรจำไปทั้งชีวิต

41 คู่ที่ "cointegrate" จากข้อมูลที่ไม่มีอะไรเลย · ถ้าคุณสแกน 100 เทริยญ (4,950 คู่) บนข้อมูลไร้รูปแบบ คุณจะได้คู่ที่น่าสนใจราว 250 คู่ ให้เลือกเทรด

นี่คือคำตอบว่าทำไมคนที่เขียน scanner เองครั้งแรกเจอโอกาสเต็มจอ แต่เทรดจริงไม่ได้สักคู่ · scanner ไม่ได้พังมันทำงานตามที่สั่งเป๊ะ — เราต่างหากที่สั่งผิด

🚫 กฎ: พื้นที่ที่คุณทดสอบมากกว่า 1 ครั้ง เกณฑ์ $p < 0.05$ ก็ใช้ไม่ได้อีกต่อไป — และการสแกนหาคุณคือการทดสอบหลายร้อยครั้งพร้อมกันโดยนิยาม

🔵 เรื่องนี้เล่มนี้สอนไปแล้ว — แต่ยังไม่ได้อามาใช้ตรงนี้

หัวข้อ 28.5 (Red Flag #5 — P-Hacking และ Multiple Testing) อธิบายกลไกนี้ไว้ครบแล้ว และ Part V ยกเกณฑ์ Harvey-Liu-Zhu ($t \geq 3$) มาให้ด้วย

สิ่งที่บัพนี้เพิ่มคือการเอามันมาใช้ ณ จุดที่มันสำคัญที่สุด — ตอนคัดคู่ · เพราะถ้าปล่อยผีผ่านด่านนี้ไป ทุกอย่างหลังจากนี้ (backtest · walk-forward · ระบบสด) ก็คือการลงแรงประณิตกับของที่ไม่มีอยู่จริง

👉 ขั้นที่ 3 — คุณผีด้วย 2 วิธี แล้วเทียบกัน

```
m = len(pairs) # 435

# ① Bonferroni — เข้มที่สุด: หารเกณฑ์ด้วยจำนวนครั้งที่ทดสอบ
bonf = res[res.pvalue < 0.05 / m]

# ② Benjamini-Hochberg (FDR) — ยอมให้มีผีปนได้บ้างในสัดส่วนที่คุณไว้
r = res.copy()
r["rank"] = np.arange(1, m + 1)
r["bh"] = 0.05 * r["rank"] / m
passed = r[r.pvalue <= r.bh]
cut = passed["rank"].max() if len(passed) else 0
fdr = r[r["rank"] <= cut]

for name, df in [("ไม่ปรับอะไร", res[res.pvalue < 0.05]),
                 ("Bonferroni", bonf), ("FDR/BH", fdr)]:
    real = sum(1 for _, x in df.iterrows() if (x.a, x.b) in REAL)
    print(f"{name:14s} เหลือ {len(df):>2} คู่ · จริง {real}/3 · ผี {len(df)-real}")
```

► OUTPUT

```
ไม่ปรับอะไร      เหลือ 15 คู่ · จริง 3/3 · ผี 12
Bonferroni       เหลือ  2 คู่ · จริง 2/3 · ผี  0
FDR/BH           เหลือ  2 คู่ · จริง 2/3 · ผี  0
```

🔴 จุดตัดสินใจที่ 2 — ไม่มีทางเลือกที่ได้ทุกอย่าง

คู่อลัมน์ "จริง" ให้ดี · การคุมผีได้หมด แลกมาด้วยการทิ้งคู่จริงไป 1 ใน 3 คู่ — COIN10/COIN11 ที่ $p = 2.6e-03$ เป็นคู่จริงแท้ ๆ แต่ไม่แข็งแกร่งจะผ่านด่านที่เข้มขึ้น

นี่ไม่ใช่ข้อบกพร่องของวิธี แต่เป็นกฎแลกเปลี่ยนที่หนีไม่พ้น: เข้มขึ้น = ผีน้อยลง + ของจริงหลุดมากขึ้น · หลวมขึ้น = ได้ของจริงครบ + ผีเต็มมือ

แล้วเลือกอะไร? ในสาย stat arb คำตอบค่อนข้างชัด: **ยอมทิ้งของจริงดีกว่ายอมรับผี** · เพราะคู่จริงที่หลุดไปแค่ทำให้เสียโอกาส แต่ผีที่หลุดเข้ามาจะ *กินเงินจริงทุกวันจนกว่าจะรู้ตัว* · และคู่จริงที่แข็งแกร่งจะกลับมาผ่านด่านเองในการสแกนรอบหน้าเมื่อข้อมูลยาวขึ้น

👉 ขั้นที่ 4 — ด้านสุดท้าย: เทรดได้จริงไหม

ผ่านด่านสถิติแล้วแปลว่า *"ความสัมพันธ์น่าจะมีจริง"* · ยังไม่ได้แปลว่า *"เทรดแล้วได้เงิน"* — เป็นคนละคำถามกันคนละชั่วตามที่ **หัวข้อ 25.4b** ว่าไว้

```
# สองตัวเลขที่ตัดสินว่าคุณเทรดได้จริงไหม
def profile(a, b):
    la, lb = np.log(prices[a]), np.log(prices[b])
    beta = sm.OLS(la, sm.add_constant(lb)).fit().params.iloc[1]
    spread = la - beta * lb

    # half-life: spread กลับเข้าค่ากลางครึ่งทางใช้เวลากี่วัน (บทที่ 14.4)
    d = spread.diff().dropna()
    lag = spread.shift(1).dropna().loc[d.index]
    k = -sm.OLS(d, sm.add_constant(lag)).fit().params.iloc[1]
    half_life = np.log(2) / k if k > 0 else np.inf

    return half_life, spread.std()

COST = 0.004 # ไป-กลับ สองขา ≈ 0.4% (บทที่ 25)

fdr[["half_life", "sd"]] = [profile(x.a, x.b) for _, x in fdr.iterrows()]
fdr["edge_2sd"] = 2 * fdr.sd # ถ้าไรคาดหวังถ้าเข้าที่ 2σ
fdr["ผ่าน hl"] = fdr.half_life.between(5, 60) # ไม่เร็วไป ไม่ช้าไป
fdr["ผ่าน edge"] = fdr.edge_2sd > COST * 2 # ต้องกว้างกว่าต้นทุน 2 เท่า

print(fdr[["a", "b", "pvalue", "half_life", "edge_2sd", "ผ่าน hl", "ผ่าน edge"]
      .to_string(index=False, formatters={"pvalue": "{:.2e}".format,
      "half_life": "{:.1f}".format, "edge_2sd": "{:.2%}".format}))
```

► OUTPUT

a	b	pvalue	half_life	edge_2sd	ผ่าน hl	ผ่าน edge
COIN05	COIN06	1.23e-08	4.1	2.50%	False	True
COIN00	COIN01	1.24e-05	8.5	4.46%	True	True

คู่ที่ "แข็งแกร่งทางสถิติ" กลับตรรกะ

COIN05/COIN06 มี $p = 1.23e-08$ — **แข็งแกร่งในทั้ง 435 คู่** · แต่ตกด้านสุดท้ายเพราะ **half-life 4.1 วัน**

แปลว่า spread เด็งกลับเร็วมาก — ซึ่งฟังดูดีแต่บนแท่งรายวันแปลว่าเราได้สัญญาณดีและเทรดบ่อย · เทรดบ่อย = จ่ายค่าธรรมเนียม 0.4% บ่อย ในขณะที่ spread กว้างแค่ 2.5% ที่ 2σ

🚫 half-life ต้องเข้ากับความถี่ของข้อมูลที่คุณมี · half-life 4 วันบนแท่งรายวันคือ "เร็วเกินกว่าที่จะตามทัน" · ถ้าอยากเทรดคุณนี่จริงต้องย้ายไปแท่งรายชั่วโมงและคิดต้นทุนใหม่ทั้งหมด — คนละโปรเจกต์กัน

✅ ผลของทั้งบท

```
print(f"กรวยคัดกรอง: {m} คู่ → 15 (p<0.05) → {len(fdr)} (FDR) → {len(final)} (เศรษฐกิจ)")
print("คู่ที่รอด:", ", ".join(f"{x.a}/{x.b}" for _, x in final.iterrows()))
```

► OUTPUT

```
กรวยคัดกรอง: 435 คู่ → 15 (p<0.05) → 2 (FDR) → 1 (เศรษฐกิจ)
คู่ที่รอด: COIN00/COIN01
```

✅ เช็คว่ามาถูกทาง — และเหลือคู่เดียวคือเรื่องปกติ

435 → 1 · คู่ที่รอดเป็นคู่จริง และรอดเพราะผ่านทั้งด้านสถิติและด้านเศรษฐกิจ ไม่ใช่เพราะโชค

ถ้าคุณรันแล้วเหลือ 10–20 คู่ อย่าดีใจ — ให้กลับไปดูว่าล้มคัม multiple testing หรือเปล่า · ถ้าเหลือ 0 คู่ ก็ไม่ใช่ความล้มเหลว นั่นคือคำตอบว่า *universe* นี้ช่วงเวลานี้ไม่มีอะไรให้เทรด — ซึ่งเกิดขึ้นบ่อยกว่าที่ใครอยากยอมรับ

🔵 [รอบสอง] สิ่งที่ยังไม่ได้ทำในบทนี้ และทำไมถึงยังไม่ได้ทำ

- ยังไม่ได้แบ่ง train/test — ทั้งบทนี้ใช้ข้อมูลทั้งชุด · จงใจ เพราะการคัดคู่กับการตรวจคู่เป็นคนละงาน · บทที่ 38 จะแบ่งข้อมูลก่อนและอะไรทั้งสิ้น แล้วเอาคู่ที่รอดมาตรวจซ้ำบนช่วงที่ยังไม่เคยเห็น
- ยังไม่มี backtest สักครั้ง — จงใจเช่นกัน · การ backtest คู่ที่ยังไม่ผ่านด่านพวกนี้คือการเสียเวลาไปกับของที่รู้อยู่แล้วว่าไม่รอด
- ยังไม่ได้ดูว่า β เสถียรไหม — `profile()` ใช้ β ตัวเดียวจากทั้งช่วง ซึ่งซ่อนสมมติฐานว่าความสัมพันธ์ไม่เปลี่ยน · บทที่ 38 จะเอา Kalman (17.2) มาตรวจข้อนี้

ลำดับสำคัญกว่าเครื่องมือ — คนส่วนใหญ่กระโดดไป backtest ตั้งแต่เจอคู่แรกที่ p สวย ๆ แล้วใช้เวลาสองสัปดาห์จนกลยุทธ์ให้กับคู่ที่เป็นผีมาตั้งแต่ต้น

📁 สถานะโปรเจกต์ ณ จบบทที่ 37

```
statarb/
├── data/
│   └── prices_30coins_2023.parquet    # จากบทที่ 36
```



```
└─ research/
  └─ 01_scan_pairs.py          # ← บทนี้
  └─ shortlist.csv            # ← ผลลัพธ์ของบทนี้
    └─ log.md                 # ← บันทึกว่าตัดคูไหนออกเพราะอะไร
└─ strategy/
└─ backtest/
└─ requirements.txt
```

`shortlist.csv` ต้องเก็บ**ทุกคู่ที่ทดสอบ** ไม่ใช่เฉพาะคู่ที่ผ่าน · พร้อมคอลัมน์ว่าตกด้านไหน — เพราะบทที่ 43 จะต้องนับว่าเราทดสอบไปทั้งหมดกี่ครั้ง (multiple testing ไม่ได้จบที่บทนี้ มันสะสมไปตลอดทั้งโปรเจกต์)

➡ บทถัดไป

บทที่ 38 · ตรวจคู่ที่รอด — ก่อนแตะ backtest แม้แต่นิดเดียว

แบ่ง in-sample/out-of-sample ก่อนทำอะไรทั้งสิ้น · β เสถียรไหม (rolling OLS เทียบ Kalman) · spread ยังอยู่ในกรอบเดิมไหม · และเราจะทิ้งคู่ที่ไม่ผ่านจริง ๆ พร้อมเขียนเหตุผลลง log